

第十一次上机作业

学号：241830137 姓名：朱子健

2026 年 5 月 20 日

1 问题描述

1.1

数对于“蒲丰投针”问题编写程序进行模拟实现。进行大量蒲丰投针实验并估计 π 的值，并写出实验次数与误差的关系。

1.2

分别用随机投点法和样本平均法计算定积分

$$I = \int_0^1 e^{-x} dx$$

的估计值，并于定积分的精确值比较，同时画出该定积分的面积区域。

2 算法原理

2.1 蒲丰投针

通过将针投到画有一组等间隔平行线的地面来统计针与线条相交的频率，从而运用已知的数学关系式计算出圆周率 π 的值。

2.2 随机投点法

随机投点法的英文名称“Hit or Miss method”，直接翻译为“命中或未命中法”。定积分的几何意义是计算一个几何区域的面积或者体积。因此利用随机投点是否落在所求区域内的信息来计算定积分是可能的。我们首先以随机投点法来介绍蒙特卡罗数值求积方法。

考虑如下定积分计算问题：

$$I = \int_a^b f(x) dx,$$

其中被积函数满足：

$$0 \leq f(x) \leq M, \quad \forall x \in [a, b],$$

M 是一个常数。

为计算上面的积分 I ，我们取一个矩形区域 A ：

$$A = \{(x, y) \mid a \leq x \leq b, 0 \leq y \leq M\}.$$

那么，由被积函数 $f(x)$ 的曲线与 x 坐标轴所围出的区域 B 为

$$B = \{(x, y) \mid a \leq x \leq b, 0 \leq y \leq f(x)\},$$

该区域的面积就是所求定积分的值 I 。

考虑随机地向矩形 A 投点 (x, y) ，则点 (x, y) 落入 B 的机会（频率） p 就是

$$p = \frac{|B|}{|A|} = \frac{I}{(b-a)M}.$$

2.3 样本平均法

蒙特卡罗方法又可看成是将积分

$$I = \int_a^b f(x) dx$$

表示成某个随机变量的数学期望。假定 $g_X(x)$ 为随机变量 X 的概率密度函数，满足 $g_X(x) > 0, \forall x \in (a, b)$ 。则上述积分可以表示为如下的数学期望：

$$I = E\left(\frac{f(x)}{g_X(x)}\right).$$

(4.8.13)

最简单的做法是假设 X 为 (a, b) 上服从均匀分布的随机变量，即

$$g_X(x) = \begin{cases} \frac{1}{b-a}, & a < x < b, \\ 0, & \text{其他.} \end{cases}$$

于是有

$$I = (b-a)E(f(X)).$$

(4.8.14)

此时，我们只要产生 N 个在 (a, b) 上服从均匀分布的随机数 $X_1, X_2, \dots, X_N$ ，并计算样本均值得到积分估计值 $\hat{I}$ ：

$$\hat{I} = (b-a) \frac{1}{N} \sum_{i=1}^N f(X_i).$$

(4.8.15)

这就是所谓的蒙特卡罗积分法中样本平均值法，也称为期望法。

算法 1 蒲丰投针

输入: 投针次数 N , 针长 h , 线间距 $d = 2h$

输出: π 的估计值 $\hat{\pi}$

```
1:  $hits \leftarrow 0$ 
2: for  $i = 1$  to  $N$  do
3:    $dist \leftarrow \text{random}(0, h)$  {针中心到最近平行线的距离}
4:    $\theta \leftarrow \text{random}(0, \pi/2)$  {针与平行线的夹角}
5:    $projection \leftarrow \frac{h}{2} \sin \theta$  {针在垂直方向的投影半长}
6:   if  $projection \geq dist$  then
7:      $hits \leftarrow hits + 1$ 
8:   end if
9: end for
10:  $P \leftarrow hits/N$  {相交频率}
11:  $\hat{\pi} \leftarrow 1/P$  {由  $P = 1/\pi$  得  $\hat{\pi} = 1/P$ }
12: return  $\hat{\pi}$ 
```

算法 2 随机投点法

输入: 投点次数 N , 积分区间 $[a, b]$, 函数 $f(x)$, 最大值 $M = \max_{x \in [a, b]} f(x)$

输出: 积分估计值 $\hat{I}$

```
1:  $hits \leftarrow 0$ 
2: for  $i = 1$  to  $N$  do
3:    $x \leftarrow \text{random}(a, b)$ 
4:    $y \leftarrow \text{random}(0, M)$ 
5:   if  $y \leq f(x)$  then
6:      $hits \leftarrow hits + 1$ 
7:   end if
8: end for
9:  $\hat{I} \leftarrow M \cdot (b - a) \cdot \frac{hits}{N}$ 
10: return  $\hat{I}$ 
```

算法 3 样本平均法

输入: 采样次数 N , 积分区间 $[a, b]$, 函数 $f(x)$

输出: 积分估计值 $\hat{I}$

```
1:  $sum \leftarrow 0$ 
2: for  $i = 1$  to  $N$  do
3:    $x_i \leftarrow \text{random}(a, b)$ 
4:    $sum \leftarrow sum + f(x_i)$ 
5: end for
6:  $\bar{f} \leftarrow sum/N$  {函数值的样本均值}
7:  $\hat{I} \leftarrow (b - a) \cdot \bar{f}$ 
8: return  $\hat{I}$ 
```

3 结果分析

表 1 展示了不同实验次数下蒲丰投针估计 π 值的结果。

实验次数 N	π 估计值	绝对误差	相对误差 (%)
100	3.076923	0.064670	2.058
500	3.115265	0.026328	0.838
1000	3.125000	0.016593	0.528
5000	3.144654	0.003061	0.097
10000	3.138365	0.003228	0.103
50000	3.142767	0.001174	0.037
100000	3.140056	0.001537	0.049

3.1 结果分析

- 随着实验次数的增加, π 的估计值逐渐收敛到理论值 3.14159。
- 当 $N = 100$ 时, 误差较大 (约 0.065); 当 $N = 50000$ 时, 误差已小于 0.002。
- 相对误差从 2.06% 下降到 0.04% 左右, 验证了蒙特卡洛方法的收敛性。
- 误差与 $\frac{1}{\sqrt{N}}$ 成正比, 符合大数定律和中心极限定理的预期。

$$I_{\text{exact}} = \int_0^1 e^{-x} dx = [-e^{-x}]_0^1 = 1 - e^{-1} \approx 0.6321205588$$

表 2 展示了两种蒙特卡洛积分方法的结果比较。

方法	估计值	绝对误差	相对误差 (%)
精确值	0.632121	-	-
随机投点法	0.631400	0.000721	0.114
样本平均法	0.632057	0.000064	0.010

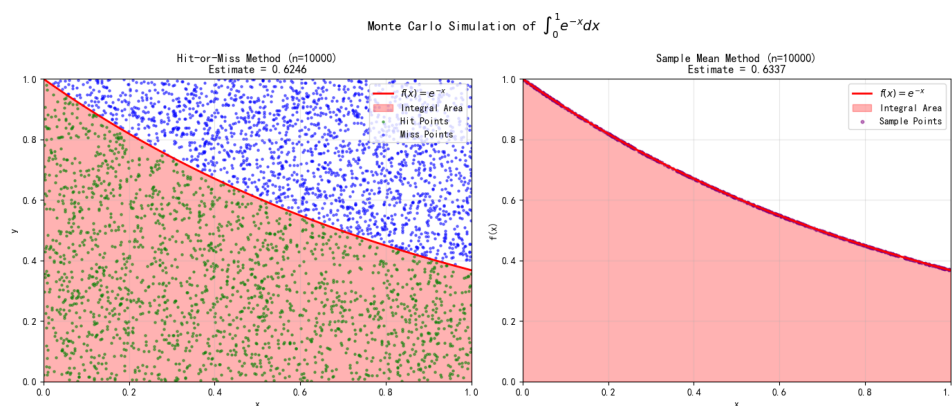


图 1: 定积分 $\int_0^1 e^{-x} dx$ 的面积区域

4 附录：程序代码

4.1

```

1 import random
2 import math
3 import numpy as np
4 import matplotlib.pyplot as plt
5 from scipy.integrate import quad
6
7 # 设置中文字体
8 plt.rcParams['font.sans-serif'] = ['SimHei', 'DejaVu Sans', 'Arial Unicode MS']
9 plt.rcParams['axes.unicode_minus'] = False
10
11 # ===== 1. 蒲丰投针问题 =====
12 def buffon_needle(num_trials, needle_length=1.0):
13     """单次蒲丰投针实验，返回 的估计值"""
14     line_spacing = 2.0 * needle_length # d = 2h
15     hits = 0
16
17     for _ in range(num_trials):
18         distance = random.uniform(0, needle_length)
19         angle = random.uniform(0, math.pi / 2.0)
20         projection = (needle_length / 2.0) * math.sin(angle)
21
22         if projection >= distance:
23             hits += 1
24
25     if hits == 0:
26         return float('inf')
27
28     probability = hits / num_trials
29     pi_estimate = 1.0 / probability
30     return pi_estimate

```

```

31
32 # 进行大量蒲丰投针实验，估计  $\pi$  的值，并记录实验次数与误差的关系
33 def buffon_needle_experiments():
34     print("=" * 60)
35     print("1. 蒲丰投针问题模拟")
36     print("=" * 60)
37
38     trial_counts = [100, 500, 1000, 5000, 10000, 50000, 100000]
39     pi_estimates = []
40     errors = []
41
42     print("\n实验次数\t估计值\t\t绝对误差")
43     print("-" * 50)
44
45     for n in trial_counts:
46         pi_est = buffon_needle(n)
47         error = abs(pi_est - math.pi)
48         pi_estimates.append(pi_est)
49         errors.append(error)
50         print(f"{n}\t\t{pi_est:.6f}\t{error:.6f}")
51
52 # ===== 2. 蒙特卡洛积分 =====
53 def f(x):
54     """被积函数  $f(x) = e^{-x}$ """
55     return np.exp(-x)
56
57 def monte_carlo_hit_or_miss(n, a=0, b=1, y_max=1):
58     """随机投点法计算定积分"""
59     hits = 0
60     hit_xs = []
61     hit_ys = []
62     miss_xs = []
63     miss_ys = []
64
65     for _ in range(n):
66         x = random.uniform(a, b)
67         y = random.uniform(0, y_max)
68
69         if y <= f(x):
70             hits += 1
71             hit_xs.append(x)
72             hit_ys.append(y)
73         else:
74             miss_xs.append(x)
75             miss_ys.append(y)
76
77     area_rect = (b - a) * y_max
78     estimate = area_rect * hits / n
79     return estimate, hit_xs, hit_ys, miss_xs, miss_ys
80

```

```

81 def monte_carlo_sample_mean(n, a=0, b=1):
82     """样本平均法计算定积分"""
83     samples = [random.uniform(a, b) for _ in range(n)]
84     func_values = [f(x) for x in samples]
85     estimate = (b - a) * sum(func_values) / n
86     return estimate, samples, func_values
87
88 def integral_experiment():
89     print("\n" + "=" * 60)
90     print("2. 蒙特卡洛积分计算  $\int_0^1 e^{-x} dx$ ")
91     print("=" * 60)
92
93     # 精确积分值
94     exact_value, _ = quad(f, 0, 1)
95
96     n_trials = 10000
97
98     # 随机投点法
99     hm_estimate, hit_xs, hit_ys, miss_xs, miss_ys = monte_carlo_hit_or_miss(
100         n_trials)
101
102     # 样本平均法
103     sm_estimate, samples, func_vals = monte_carlo_sample_mean(n_trials)
104
105     print(f"\n精确积分值: {exact_value:.6f}")
106     print(f"随机投点法估计值: {hm_estimate:.6f}")
107     print(f"绝对误差: {abs(hm_estimate - exact_value):.6f}")
108     print(f"样本平均法估计值: {sm_estimate:.6f}")
109     print(f"绝对误差: {abs(sm_estimate - exact_value):.6f}")
110
111     # 画出该定积分的面积区域
112     fig, axes = plt.subplots(1, 2, figsize=(14, 6))
113
114     # 左图: 随机投点法
115     ax1 = axes[0]
116     x_curve = np.linspace(0, 1, 1000)
117     y_curve = f(x_curve)
118
119     ax1.plot(x_curve, y_curve, 'r-', linewidth=2, label=r' $f(x) = e^{-x}$ ')
120     ax1.fill_between(x_curve, 0, y_curve, alpha=0.3, color='red', label='Integral Area')
121
122     # 画点 (只画部分避免过于密集)
123     sample_size = min(2000, len(hit_xs))
124     ax1.scatter(hit_xs[:sample_size], hit_ys[:sample_size],
125                 c='green', s=5, alpha=0.5, label='Hit Points')
126     ax1.scatter(miss_xs[:sample_size], miss_ys[:sample_size],
127                 c='blue', s=5, alpha=0.5, label='Miss Points')
128
129     ax1.set_xlim(0, 1)

```

```

129     ax1.set_ylim(0, 1)
130     ax1.set_xlabel('x')
131     ax1.set_ylabel('y')
132     ax1.set_title(f'Hit-or-Miss Method (n={n_trials}) \n Estimate = {hm_estimate:.4f}'
133                  ')
134     ax1.legend(loc='upper right')
135     ax1.grid(True, alpha=0.3)
136
137     # 右图：样本平均法
138     ax2 = axes[1]
139     ax2.plot(x_curve, y_curve, 'r-', linewidth=2, label=r'$f(x) = e^{-x}$')
140     ax2.fill_between(x_curve, 0, y_curve, alpha=0.3, color='red', label='Integral Area')
141
142     # 画采样点
143     sample_size = min(500, len(samples))
144     ax2.scatter(samples[:sample_size], func_vals[:sample_size],
145                 c='purple', s=10, alpha=0.5, label='Sample Points')
146
147     ax2.set_xlim(0, 1)
148     ax2.set_ylim(0, 1)
149     ax2.set_xlabel('x')
150     ax2.set_ylabel('f(x)')
151     ax2.set_title(f'Sample Mean Method (n={n_trials}) \n Estimate = {sm_estimate:.4f}'
152                  ')
153     ax2.legend(loc='upper right')
154     ax2.grid(True, alpha=0.3)
155
156     plt.suptitle(r'Monte Carlo Simulation of  $\int_0^1 e^{-x} dx$ ', fontsize=14)
157     plt.tight_layout()
158     plt.show()
159
160     # ===== 主程序 =====
161     if __name__ == "__main__":
162         # 1. 蒲丰投针问题
163         buffon_needle_experiments()
164
165         # 2. 蒙特卡洛积分
166         integral_experiment()

```